

CONECTASTORE — SISTEMA DE RECOMENDAÇÃO DE PRODUTOS BASEADO EM GRAFOS

RELATÓRIO EXPLICATIVO

Adriano Mantoan

RA:175928

São Paulo

Setembro de 2026

Sumário

1. Introdução	3
2. Objetivo	4
2.1 Objetivo Geral.....	4
2.2 Objetivos Específicos	4
3. Ferramentas e Estruturas de Dados Utilizadas	4
3.1 Rust e Cargo	4
3.2 GitHub e GitHub Codespaces	6
3.3 HashMap	6
3.4 Lista de Adjacência	7
3.5 Vec, VecDeque e HashSet.....	7
4. Desenvolvimento das Atividades	8
4.1 Levantamento dos Requisitos e Planejamento do Projeto	8
4.2 Modelagem do Sistema como Grafo.....	8
4.3 Definição dos Vértices, Arestas, Pesos e Tipo de Grafo	10
4.4 Comparação entre Lista e Matriz de Adjacência	11
4.5 Implementação do Catálogo de Produtos.....	12
4.6 Implementação do Grafo.....	13
4.7 Implementação do Algoritmo de Recomendação	13
4.8 Prevenção de Recomendações Duplicadas	15
4.9 Testes Unitários e de Integração	15
4.10 Testes de Desempenho e Escalabilidade	16
4.11 Versionamento e Documentação no GitHub.....	17
5. Dificuldades Encontradas	19
6. Análise dos Resultados.....	21
6.1 Funcionamento do Sistema de Recomendação	21
6.2 Análise de Complexidade.....	21
6.3 Desempenho e Escalabilidade.....	22
7. Conclusão	23
8. Referências	24
9. Links da Entrega	24

1. Introdução

Neste trabalho desenvolvi o projeto ConectaStore — Sistema de Recomendação de Produtos Baseado em Grafos, proposto na disciplina de Data Structure Strategy and Implementation.

O cenário apresentado no trabalho é o da empresa fictícia MegaStore, que possui um catálogo muito grande de produtos. O problema é que, com tantos produtos disponíveis, fica difícil para o cliente encontrar itens realmente relacionados ao seu interesse. O sistema atual faz recomendações mais genéricas, como produtos mais vendidos ou da mesma categoria, sem analisar de forma mais completa as relações entre clientes, produtos, compras e interesses.

A proposta do projeto foi justamente pensar em uma forma melhor de representar essas relações utilizando grafos. Nesse tipo de estrutura, os elementos podem ser representados como vértices e as relações entre eles como arestas.

Durante o desenvolvimento, optei por representar clientes e produtos como vértices. As conexões entre eles representam interesses, compras ou relações entre produtos.

Além do grafo, utilizei outras estruturas de dados, como HashMap, Vec, VecDeque e HashSet, cada uma com uma função específica dentro do sistema.

Para realizar as recomendações, utilizei o algoritmo de busca em largura, conhecido como BFS — Breadth-First Search. Esse algoritmo percorre o grafo por níveis e permite encontrar produtos relacionados aos itens que já estão associados ao cliente.

O objetivo principal do trabalho foi entender, na prática, como diferentes estruturas de dados podem trabalhar juntas para resolver um problema de recomendação de produtos.

2. Objetivo

2.1 Objetivo Geral

O objetivo geral deste projeto foi desenvolver, em Rust, um sistema simples de recomendação de produtos utilizando grafos.

A ideia foi representar as relações entre clientes e produtos de uma forma que permitisse ao sistema percorrer essas conexões e encontrar novos produtos que pudessem ser recomendados.

2.2 Objetivos Específicos

Durante o desenvolvimento, busquei atingir os seguintes objetivos:

- criar uma estrutura para cadastrar produtos;
- permitir a consulta de produtos por identificador;
- utilizar HashMap para facilitar essa consulta;
- representar clientes e produtos dentro de um grafo;
- criar conexões entre esses vértices;
- utilizar pesos nas conexões;
- implementar o grafo com lista de adjacência;
- utilizar BFS para percorrer o grafo;
- fazer recomendações a partir de um cliente;
- também permitir recomendações a partir de um produto;
- evitar produtos duplicados na lista de recomendações;
- criar testes para conferir se as principais funções estavam funcionando;
- medir o tempo de execução com diferentes quantidades de produtos;
- documentar o projeto no GitHub.

3. Ferramentas e Estruturas de Dados Utilizadas

3.1 Rust e Cargo

A linguagem utilizada no projeto foi Rust, conforme solicitado no trabalho.

No começo, eu ainda não tinha muita familiaridade com Rust, então precisei entender melhor como funcionavam algumas características da linguagem, principalmente organização dos módulos, tipos, estruturas, enum, referências e coleções.

Para criar, compilar, executar e testar o projeto utilizei o Cargo, que é a ferramenta utilizada pelo Rust para gerenciamento dos projetos.

Durante o desenvolvimento utilizei principalmente os comandos:

`cargo check`

para verificar se o código estava compilando corretamente;

`cargo run`

para executar o sistema;

`cargo test`

para executar os testes;

`cargo clippy`

para verificar avisos e possíveis melhorias no código.

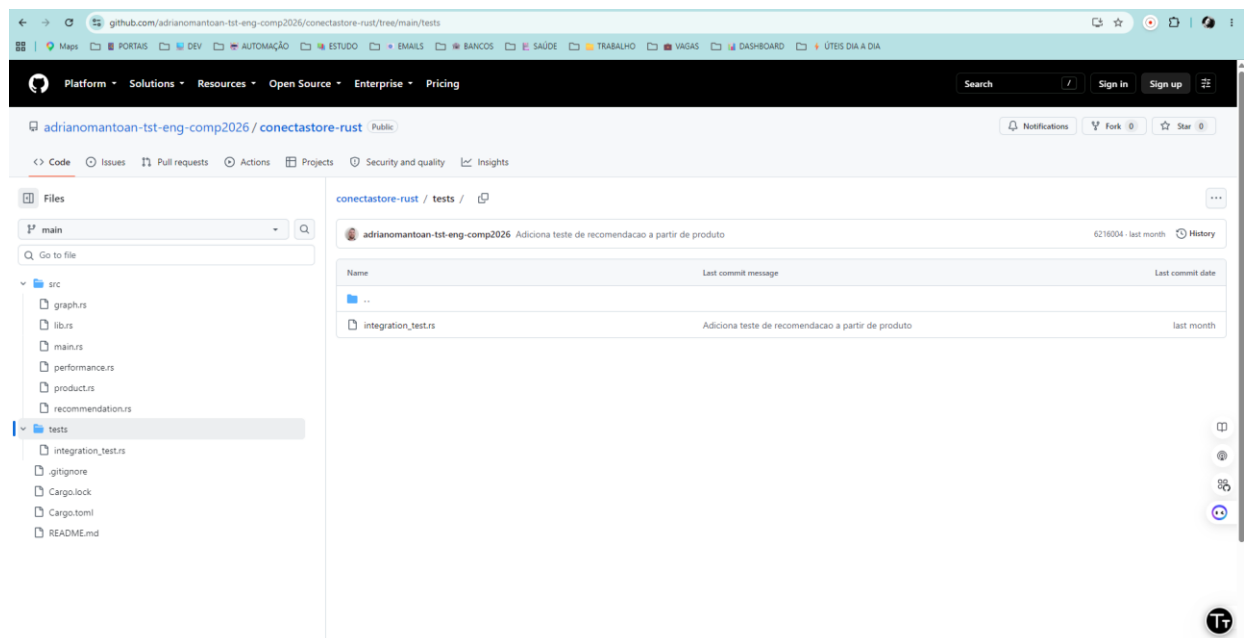


Figura 1 – Estrutura do projeto ConectaStore no ambiente de desenvolvimento.
Fonte: Elaborado pelo autor (2026).

3.2 GitHub e GitHub Codespaces

O projeto foi desenvolvido utilizando o GitHub Codespaces, diretamente pelo navegador.

No início, criei um repositório público chamado conectastore-rust e depois iniciei o projeto Rust dentro dele.

Também utilizei Git durante o desenvolvimento para salvar as diferentes etapas do projeto.

Isso foi importante porque eu não deixei para enviar todo o código apenas no final. Fui criando commits conforme novas partes eram concluídas, como o catálogo de produtos, o grafo, a recomendação BFS, os testes e a documentação.

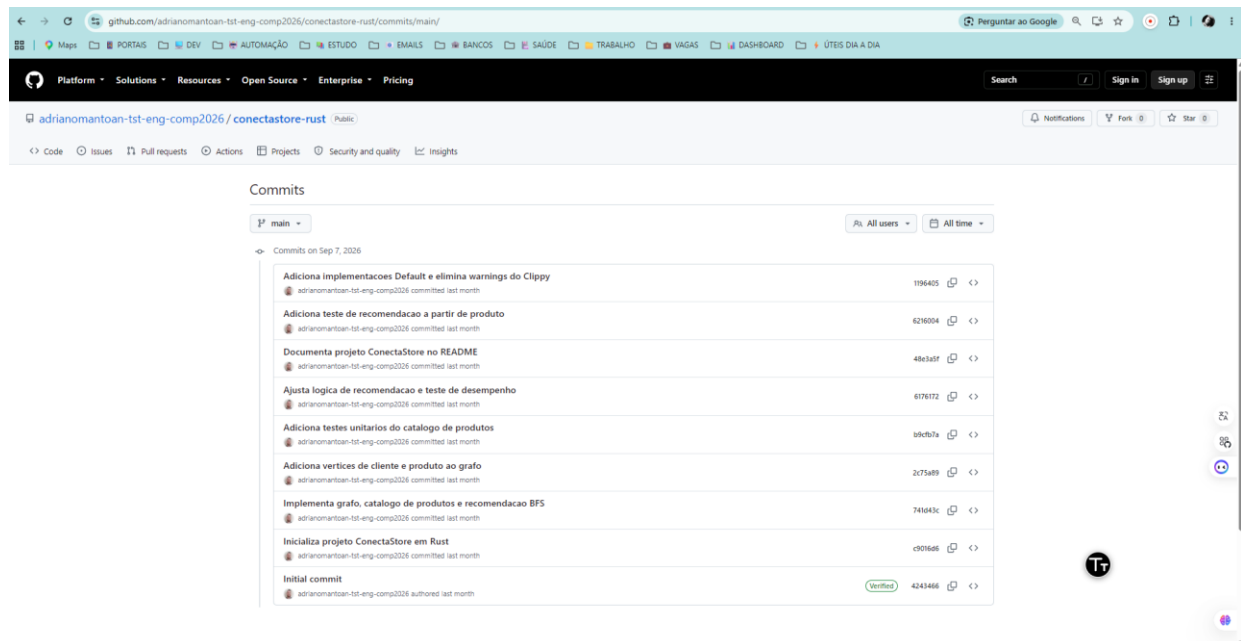


Figura 2 – Histórico de commits realizados durante o desenvolvimento.
Fonte: Elaborado pelo autor (2026).

3.3 HashMap

Uma das estruturas utilizadas foi o HashMap.

No catálogo de produtos, eu utilizei essa estrutura para relacionar o ID do produto ao próprio produto.

Por exemplo, um produto com ID 1 pode ser acessado diretamente utilizando esse identificador.

Também utilizei HashMap para implementar a lista de adjacência do grafo.

Essa escolha foi feita porque o HashMap facilita a consulta de informações por meio de uma chave.

3.4 Lista de Adjacência

Para representar o grafo, escolhi utilizar uma lista de adjacência.

Na prática, cada vértice possui uma lista contendo suas conexões.

No projeto essa estrutura ficou semelhante a:

```
HashMap<Vertex, Vec<Edge>>
```

Isso significa que cada vértice é utilizado como uma chave e possui um vetor com suas arestas.

Eu escolhi essa estrutura porque ela não precisa armazenar todas as combinações possíveis entre os vértices. Ela armazena somente as conexões que realmente existem.

3.5 Vec, VecDeque e HashSet

Além do HashMap, utilizei outras estruturas.

O Vec foi utilizado para guardar as arestas ligadas a cada vértice.

O VecDeque foi utilizado como fila durante a execução do BFS.

Já o HashSet foi utilizado para controlar os vértices que já tinham sido visitados.

Essa parte foi importante porque o mesmo produto poderia ser alcançado por mais de um caminho dentro do grafo. O HashSet evita que ele apareça várias vezes na recomendação.

4. Desenvolvimento das Atividades

4.1 Levantamento dos Requisitos e Planejamento do Projeto

Antes de começar a programar, analisei o PDF da atividade para entender exatamente o que precisava ser entregue.

Os principais pontos eram desenvolver o sistema em Rust, utilizar grafos, permitir recomendações a partir de um cliente ou produto, utilizar alguma estrutura complementar como HashMap, evitar recomendações duplicadas, criar testes e medir o tempo de execução com diferentes volumes.

Com isso, resolvi dividir o projeto em arquivos separados para facilitar a organização.

A estrutura criada foi:

- product.rs
- graph.rs
- recommendation.rs
- performance.rs
- main.rs
- lib.rs
- pasta tests

Essa divisão me ajudou porque cada arquivo ficou responsável por uma parte do sistema.

4.2 Modelagem do Sistema como Grafo

Depois de organizar o projeto, comecei a pensar em como representar o problema utilizando grafos.

Decidi utilizar dois tipos principais de vértices:

- cliente;
- produto.

O cliente representa o usuário que receberá as recomendações.

O produto representa os itens disponíveis no catálogo.

As conexões entre cliente e produto representam produtos que já fazem parte do interesse ou do histórico daquele cliente.

Já as conexões entre produtos representam itens relacionados entre si.

Um exemplo simples ficou assim:

Cliente 100 se relaciona com Produto 1 e Produto 2.

Produto 1 se relaciona com Produto 3.

Produto 2 se relaciona com Produto 4.

Dessa forma, os Produtos 1 e 2 representam os interesses já conhecidos do cliente, enquanto os Produtos 3 e 4 podem ser encontrados pelo sistema como novas recomendações.

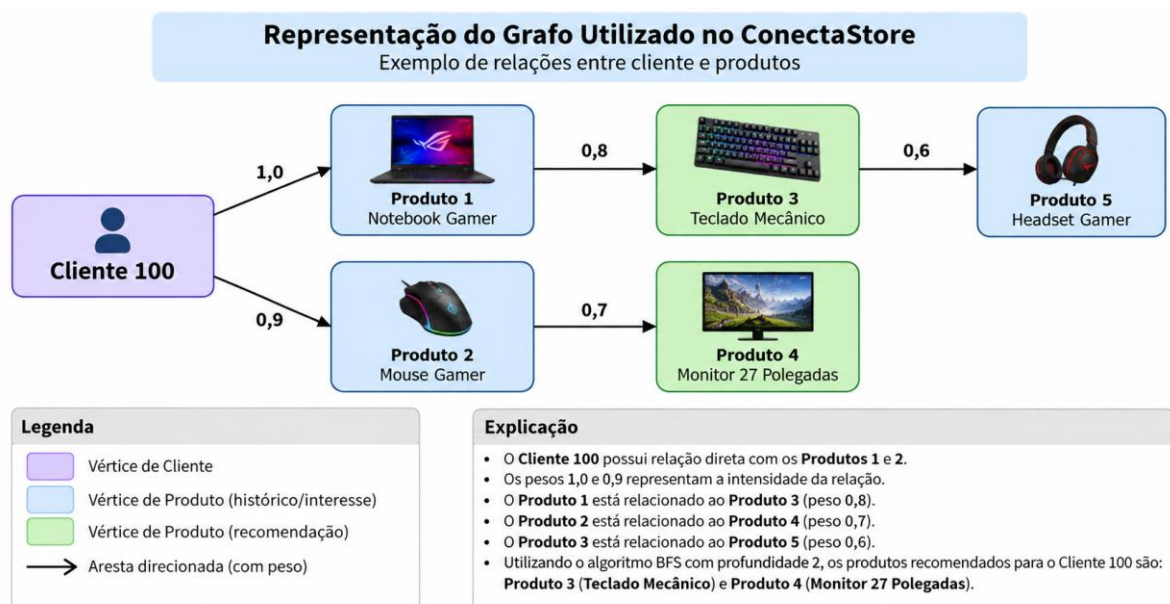


Figura 3 – Exemplo da relação entre cliente e produtos no grafo.

Fonte: Elaborado pelo autor (2026).

4.3 Definição dos Vértices, Arestas, Pesos e Tipo de Grafo

No início, eu tinha criado os vértices utilizando somente números do tipo u32.

Depois percebi que isso poderia gerar confusão.

Por exemplo, poderia existir um Cliente 1 e também um Produto 1.

Para melhorar a estrutura, criei um enum chamado Vertex.

Ele possui duas possibilidades:

Vertex::Client

e

Vertex::Product

Com isso, mesmo que os dois tenham o mesmo número de identificação, o programa consegue diferenciar os tipos.

Também criei a estrutura Edge.

Cada aresta possui um destino e um peso.

O peso foi criado para representar a intensidade da relação entre os vértices.

No sistema atual, a BFS utiliza principalmente a distância em níveis para encontrar os produtos. Mesmo assim, mantive o peso nas arestas porque ele faz parte da modelagem e pode futuramente ser utilizado para priorizar produtos com relações mais fortes.

O grafo é direcionado, porque uma conexão é criada da origem para o destino.

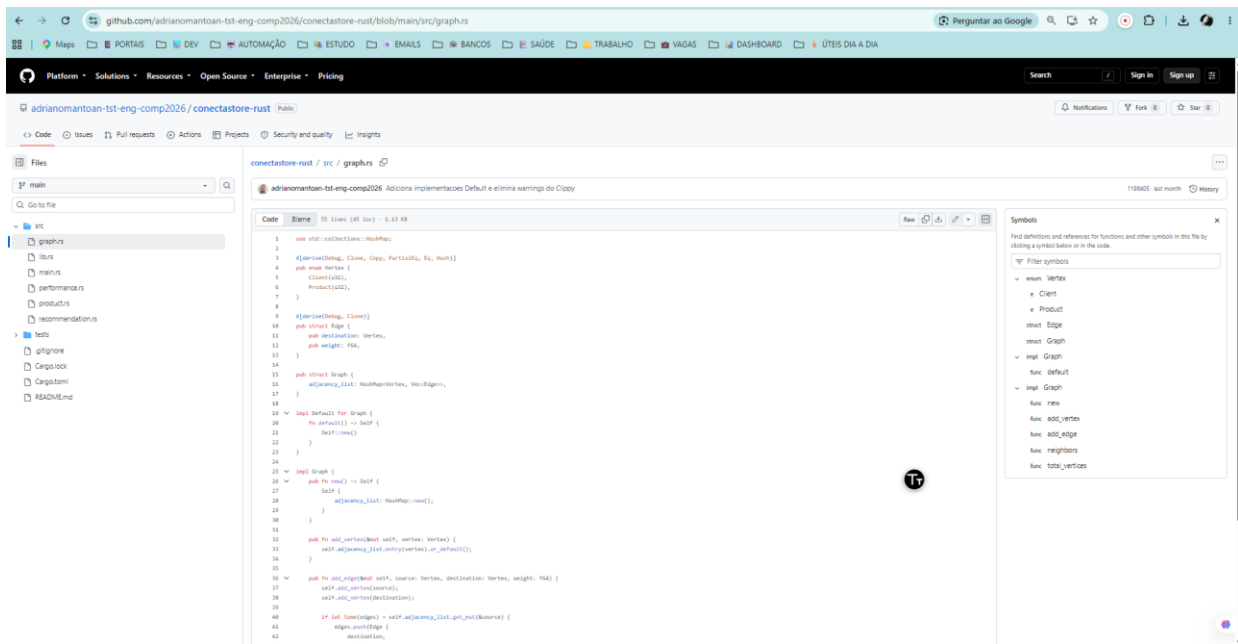


Figura 4 – Implementação dos vértices, arestas e grafo.

Fonte: Elaborado pelo autor (2026).

4.4 Comparação entre Lista e Matriz de Adjacência

Durante o trabalho também foi necessário comparar a lista de adjacência com a matriz de adjacência.

A matriz de adjacência representa todas as possibilidades de conexão entre os vértices.

Se existirem muitos vértices, a quantidade de posições cresce muito.

Em um catálogo com milhares ou milhões de produtos, isso pode representar um consumo muito maior de memória.

Já a lista de adjacência armazena apenas as relações que realmente existem.

Por isso considerei essa opção mais adequada para o projeto.

Critério	Matriz de Adjacência	Lista de Adjacência
Uso de memória	Maior em grafos grandes	Menor em grafos esparsos
Espaço	$O(V^2)$	$O(V + E)$
Percorrer vizinhos	Precisa verificar várias posições	Percorre somente conexões existentes
Escolha no projeto	Não	Sim

4.5 Implementação do Catálogo de Produtos

Depois da modelagem inicial, desenvolvi a estrutura responsável pelos produtos.

Criei uma estrutura chamada Product com quatro informações:

- ID;
- nome;
- categoria;
- preço.

Depois criei o ProductCatalog.

Esse catálogo utiliza um HashMap para armazenar os produtos.

Foram criadas funções para adicionar um produto, consultar um produto por ID e verificar a quantidade total de produtos cadastrados.

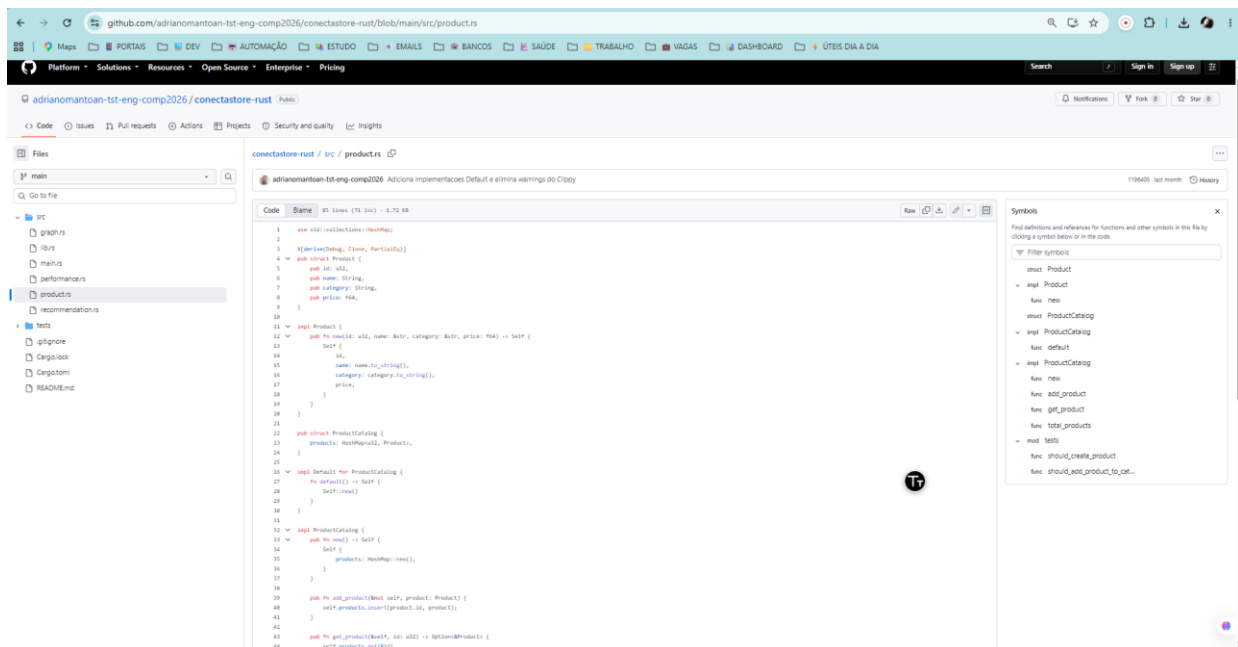


Figura 5 – Implementação do cadastro e consulta dos produtos.

Fonte: Elaborado pelo autor (2026).

4.6 Implementação do Grafo

A etapa seguinte foi implementar o próprio grafo.

A estrutura Graph guarda uma lista de adjacência.

Foram criados métodos para adicionar vértices, adicionar arestas e consultar os vizinhos de um determinado vértice.

Quando uma aresta é criada, o programa também garante que os dois vértices existam dentro do grafo.

Essa parte foi a base para depois implementar o algoritmo de recomendação.

4.7 Implementação do Algoritmo de Recomendação

Para fazer a busca das recomendações utilizei o algoritmo BFS.

No começo eu precisei entender melhor como esse algoritmo funcionava.

A ideia principal é percorrer o grafo por níveis.

Para isso, utilizei uma fila com VecDeque.

O primeiro elemento da fila é processado, seus vizinhos são analisados e os novos elementos são colocados no final da fila.

Esse processo continua até atingir a profundidade definida.

No caso da recomendação iniciada por um cliente, os produtos diretamente ligados a ele são considerados produtos que ele já conhece ou já demonstrou interesse.

O sistema então procura produtos conectados a esses produtos iniciais.

No exemplo principal, o Cliente 100 está relacionado aos Produtos 1 e 2.

A BFS encontra os Produtos 3 e 4 e apresenta esses produtos como recomendação.

Também foi criado um teste para confirmar que o mesmo algoritmo funciona quando a busca começa diretamente por um produto.

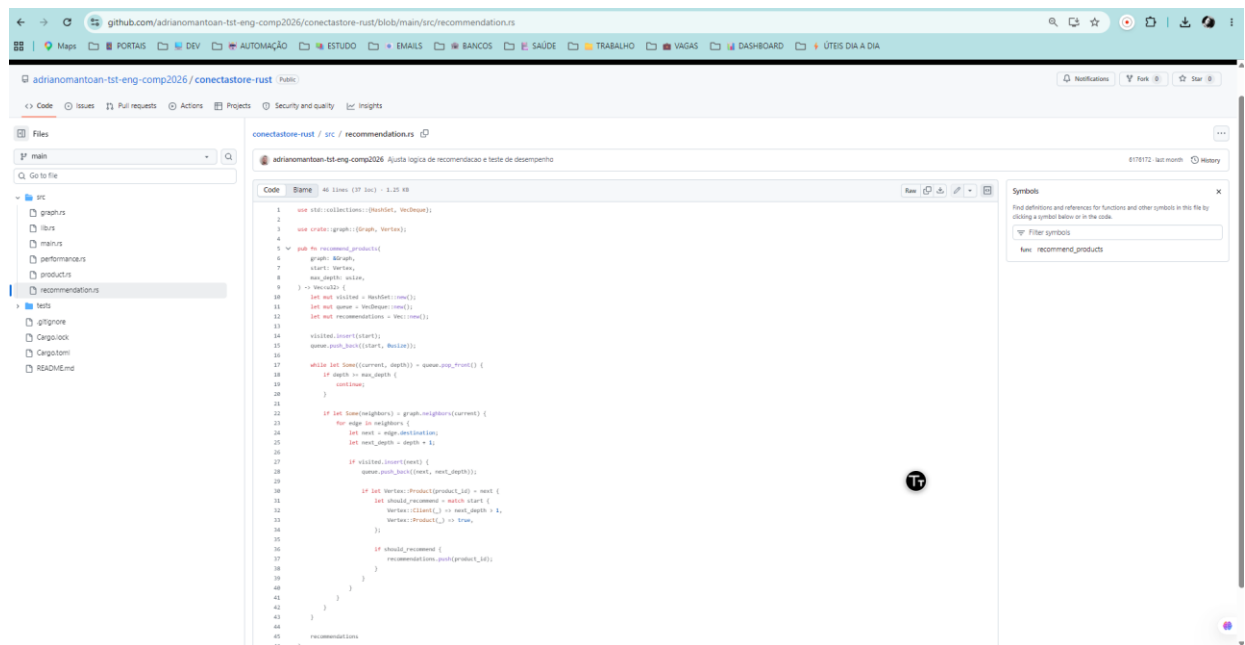


Figura 6 – Algoritmo BFS utilizado na geração das recomendações.

Fonte: Elaborado pelo autor (2026).

4.8 Prevenção de Recomendações Duplicadas

Durante o desenvolvimento percebi que um mesmo produto poderia ser encontrado por caminhos diferentes.

Por exemplo, dois produtos diferentes poderiam estar conectados ao mesmo produto recomendado.

Se não existisse nenhum controle, ele poderia aparecer mais de uma vez no resultado.

Para evitar isso utilizei um HashSet.

Sempre que um vértice é encontrado, ele é registrado como visitado.

Se ele já tiver sido visitado, o algoritmo não o adiciona novamente.

Isso também ajuda a evitar problemas caso existam ciclos dentro do grafo.

4.9 Testes Unitários e de Integração

Depois das funcionalidades principais, comecei a criar os testes.

Foram criados dois testes unitários relacionados aos produtos.

Um deles verifica a criação de um produto.

O outro verifica a inserção de um produto no catálogo.

Também foram criados três testes de integração.

Esses testes verificam:

- cadastro e consulta de produto;
- recomendação sem duplicidades;
- recomendação iniciada a partir de um produto.

Ao executar cargo test, todos os cinco testes foram aprovados.

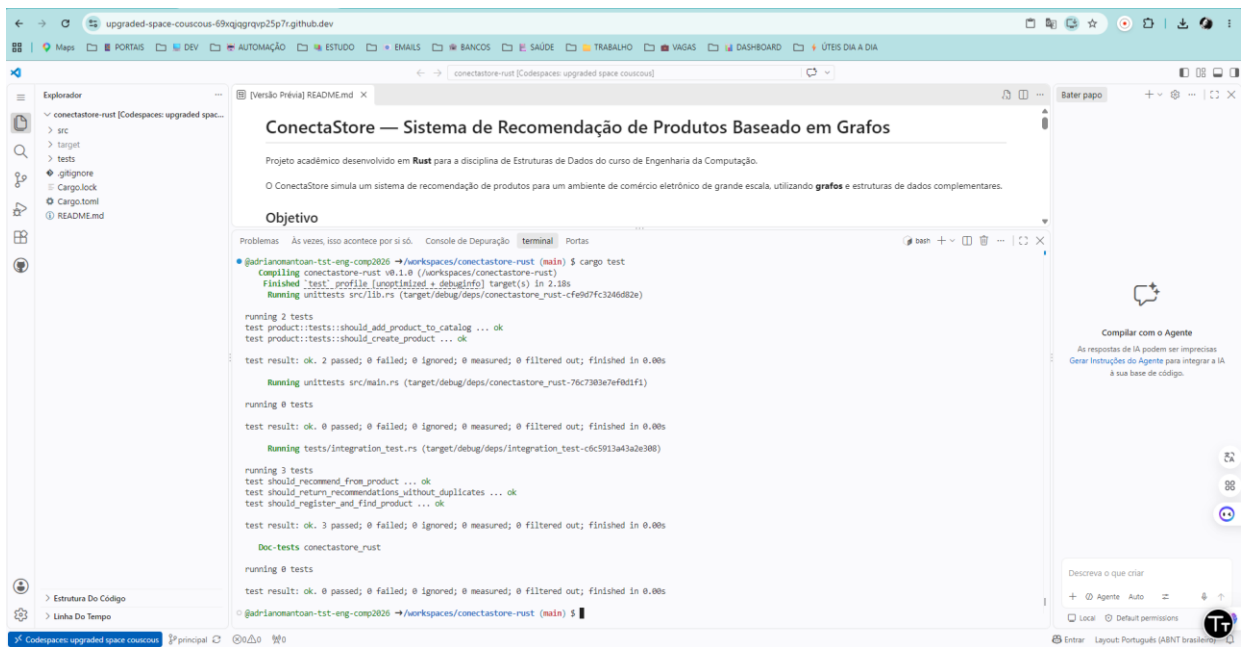


Figura 7 – Execução dos testes unitários e de integração do ConectaStore.

Fonte: Elaborado pelo autor (2026).

4.10 Testes de Desempenho e Escalabilidade

Outra exigência do projeto era realizar uma medição simples do tempo de execução com diferentes volumes de dados.

Para isso criei um módulo chamado performance.rs.

Utilizei std::time::Instant para medir quanto tempo a busca levava.

Foram executados testes com:

1.000 produtos;

10.000 produtos;

50.000 produtos.

Em uma das execuções obtive os seguintes resultados:

Produtos	Recomendações	Tempo
1.000	999	1,293865 ms
10.000	9.999	16,394195 ms
50.000	49.999	47,091753 ms

É importante destacar que esses tempos podem mudar de uma execução para outra.

Mesmo assim, o teste permitiu observar o comportamento do sistema quando a quantidade de dados aumentou.

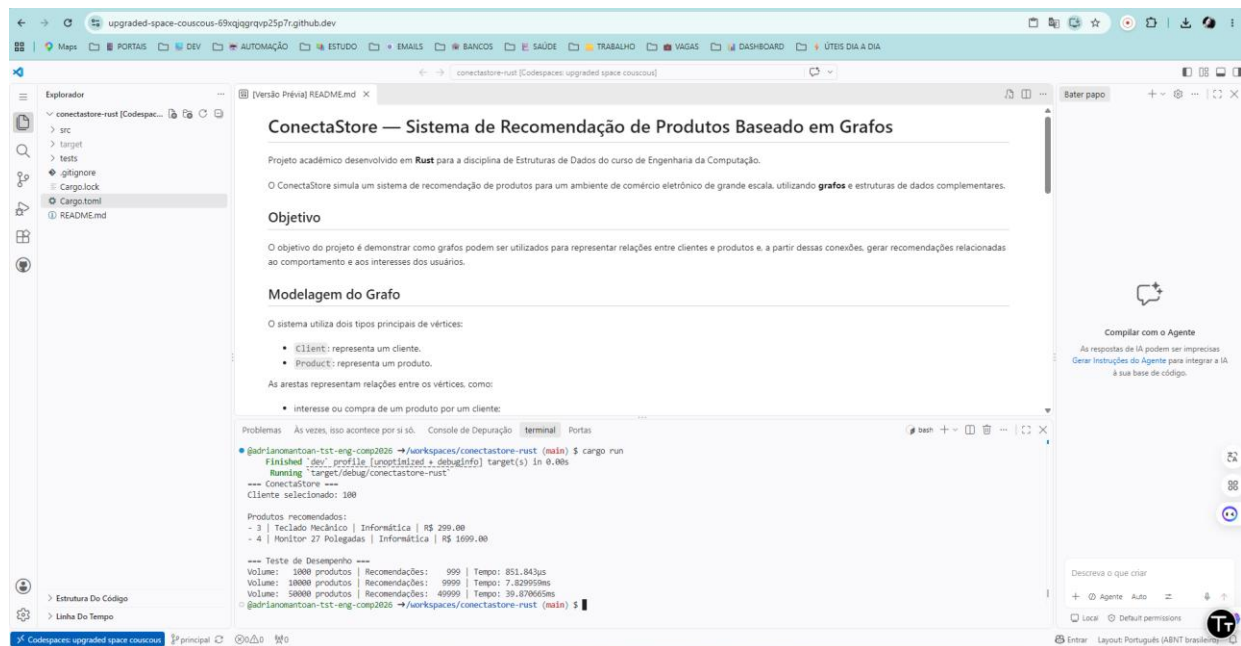


Figura 8 – Resultado da execução e dos testes de desempenho.

Fonte: Elaborado pelo autor (2026).

4.11 Versionamento e Documentação no GitHub

Durante o desenvolvimento fui realizando commits para registrar as etapas concluídas.

No total, foram feitos commits específicos para:

- inicialização do projeto;
- implementação do grafo;
- catálogo de produtos;
- BFS;
- diferenciação entre clientes e produtos;
- testes unitários;
- ajustes da recomendação;
- testes de desempenho;
- README;
- teste de recomendação por produto;
- correções do Clippy.

Também foi criado o arquivo README.md.

Nesse arquivo registrei o objetivo do projeto, as estruturas utilizadas, como executar o sistema, como executar os testes e os resultados do benchmark.

Repositório:

<https://github.com/adrianomantoan-tst-eng-comp2026/conectastore-rust>

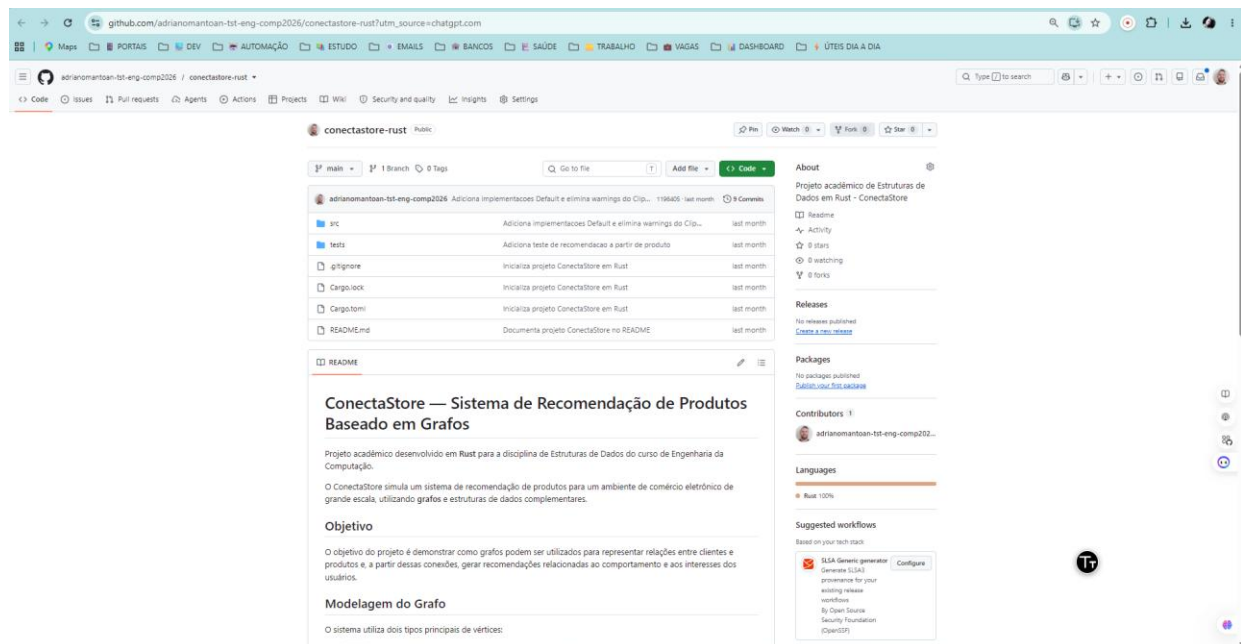


Figura 9 – Repositório público do projeto ConectaStore.

Fonte: Elaborado pelo autor (2026).

5. Dificuldades Encontradas

Durante o desenvolvimento encontrei algumas dificuldades, principalmente porque ainda estou aprendendo Rust e também estou aprofundando meus conhecimentos sobre estruturas de dados.

Uma das primeiras dificuldades foi configurar o ambiente.

Quando executei o comando para verificar o Rust, percebi que ele ainda não estava instalado no Codespaces.

Então precisei instalar a linguagem e configurar o ambiente antes de iniciar o projeto.

Outra dificuldade aconteceu durante a criação do primeiro arquivo.

Em um momento, acabei colando código Rust diretamente no terminal e o Bash tentou interpretar comandos como `pub`, `impl` e `Self`.

Depois percebi que o código deveria ser colocado dentro do arquivo `.rs`, e não diretamente no terminal.

Durante a modelagem do grafo também precisei fazer alterações.

Inicialmente eu estava utilizando apenas números para identificar os vértices.

Depois percebi que seria melhor diferenciar clientes e produtos.

Foi então que criei o enum `Vertex`.

Outra dificuldade ocorreu na lógica da recomendação.

Na primeira versão, os produtos que já estavam ligados diretamente ao cliente também apareciam como recomendações.

Depois de analisar melhor o funcionamento, percebi que esses produtos deveriam representar o histórico ou interesse do cliente e não novas recomendações.

Fiz então uma alteração para que o sistema recomendasse somente os produtos encontrados nos níveis seguintes.

Essa alteração também afetou o teste de desempenho.

Como todos os produtos estavam ligados diretamente ao cliente, a quantidade de recomendações passou a ser zero.

Foi necessário modificar o benchmark e criar um produto inicial de interesse, ligado aos demais produtos.

Por fim, utilizei o cargo clippy para verificar possíveis melhorias.

Ele mostrou dois avisos relacionados à implementação de Default nas estruturas ProductCatalog e Graph.

Realizei as correções e, depois disso, o Clippy executou sem nenhum warning.

Essas dificuldades fizeram parte do processo de desenvolvimento e me ajudaram a entender melhor tanto a linguagem Rust quanto o funcionamento das estruturas utilizadas.

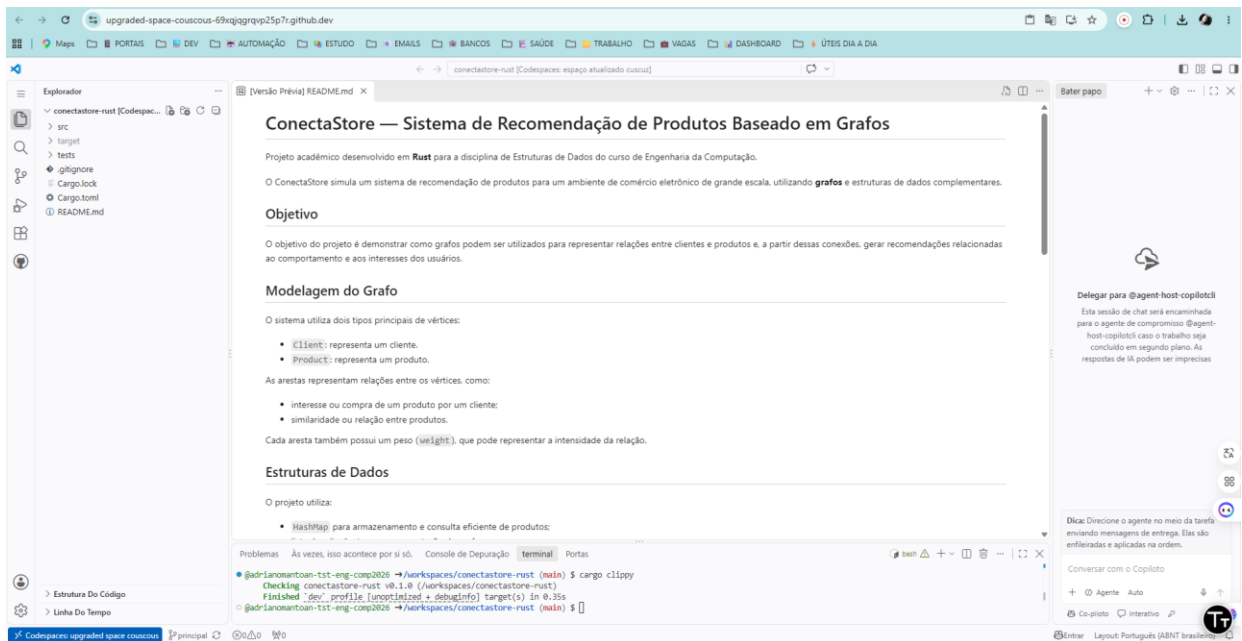


Figura 10 – Verificação final do projeto utilizando Cargo Clippy.

Fonte: Elaborado pelo autor (2026).

6. Análise dos Resultados

6.1 Funcionamento do Sistema de Recomendação

Depois das correções e dos testes, o sistema conseguiu executar o comportamento esperado.

No exemplo principal foi utilizado o Cliente 100.

Esse cliente possui ligação direta com dois produtos.

Esses produtos representam itens que já fazem parte do interesse do cliente.

A partir deles, o sistema percorre o grafo e encontra novos produtos relacionados.

O resultado apresentado na execução foi:

- Teclado Mecânico;
- Monitor 27 Polegadas.

Esses produtos foram encontrados a partir das relações existentes no grafo.

Também consegui confirmar através dos testes que produtos repetidos não aparecem duas vezes e que a recomendação pode começar diretamente por um produto.

6.2 Análise de Complexidade

Uma parte importante do trabalho foi entender a complexidade do algoritmo.

A BFS utilizando lista de adjacência possui complexidade aproximada de:

$$O(V + E)$$

onde:

V representa os vértices;

E representa as arestas.

Isso acontece porque o algoritmo percorre os vértices e as conexões que são alcançadas durante a busca.

A memória utilizada pela lista de adjacência também pode ser representada como:

$$O(V + E)$$

Já uma matriz de adjacência precisaria de:

$$O(V^2)$$

Por esse motivo, entendi que a lista de adjacência seria uma escolha mais adequada para um catálogo muito grande.

6.3 Desempenho e Escalabilidade

Nos testes realizados, o sistema conseguiu trabalhar com 1.000, 10.000 e 50.000 produtos.

O tempo aumentou conforme o volume aumentou, o que já era esperado.

Apesar disso, a execução permaneceu rápida dentro dos testes realizados.

Eu não considero esses valores como um teste profissional de desempenho, porque eles dependem do computador e do ambiente utilizado.

Mas eles serviram para demonstrar que a implementação consegue trabalhar com volumes maiores de dados e também para observar o crescimento do tempo de processamento.

Em um sistema real com milhões de produtos, provavelmente seriam necessários outros recursos, como banco de dados, armazenamento persistente, processamento distribuído e mecanismos de ranking mais avançados.

Mesmo assim, para o objetivo deste trabalho, o comportamento obtido foi satisfatório.

7. Conclusão

Com este projeto consegui colocar em prática conceitos que até então eu tinha visto mais na teoria.

O principal aprendizado foi entender como um grafo pode ser utilizado para representar relações entre diferentes elementos.

No ConectaStore, os clientes e produtos foram representados como vértices e as relações foram representadas como arestas.

Também consegui perceber que somente o grafo não seria suficiente para organizar todas as partes do sistema.

Por isso foram utilizadas estruturas complementares.

O HashMap ajudou no armazenamento e consulta dos produtos.

O Vec foi utilizado para armazenar as conexões.

O VecDeque foi utilizado como fila durante a BFS.

O HashSet foi importante para controlar os elementos visitados e evitar recomendações duplicadas.

Outro ponto importante foi perceber que a primeira solução nem sempre é a melhor.

Durante o desenvolvimento precisei revisar partes do código, principalmente a identificação dos vértices e a lógica das recomendações.

Os testes ajudaram bastante nesse processo.

Ao final, foram executados cinco testes com sucesso, além dos testes de desempenho com diferentes volumes de produtos.

Considero que o projeto atingiu o objetivo proposto, pois foi possível desenvolver um sistema em Rust que utiliza grafos e outras estruturas de dados para gerar recomendações relacionadas aos interesses de um cliente.

Além do funcionamento do programa, o trabalho também me ajudou a compreender melhor temas como BFS, listas de adjacência, HashMap, complexidade de algoritmos, testes e versionamento com Git.

Essa versão fica muito mais crível como um relatório escrito por você: mantém o conteúdo técnico necessário para a nota, mas a narrativa mostra aprendizado, tentativas, correções e entendimento progressivo.

Eu também manteria exatamente esse tom na seção de Dificuldades Encontradas, porque ela é uma das melhores partes para demonstrar que você realmente participou do desenvolvimento, em vez de parecer um texto genérico produzido por terceiros.

8. Referências

RUST FOUNDATION. Rust Programming Language. Disponível em: <https://www.rust-lang.org/>. Acesso em: set. 2026.

RUST PROJECT. The Rust Programming Language. Disponível em: <https://doc.rust-lang.org/book/>. Acesso em: set. 2026.

RUST PROJECT. Rust Standard Library Documentation. Disponível em: <https://doc.rust-lang.org/std/>. Acesso em: set. 2026.

GITHUB. GitHub Docs. Disponível em: <https://docs.github.com/>. Acesso em: set. 2026.

UNIFECAP. Data Structure Strategy and Implementation — ConectaStore: Sistema de Recomendação de Produtos Baseado em Grafos. Material da atividade acadêmica. São Paulo, 2026.

9. Links da Entrega

Repositório público do projeto:

<https://github.com/adrianomantoan-tst-eng-comp2026/conectastore-rust>

Vídeo Pitch:

<https://youtu.be/NzeDg6Usanc>